Keith Framework - Steppable

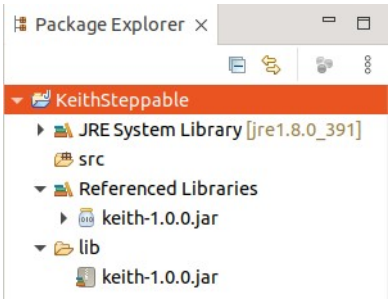
Introduction

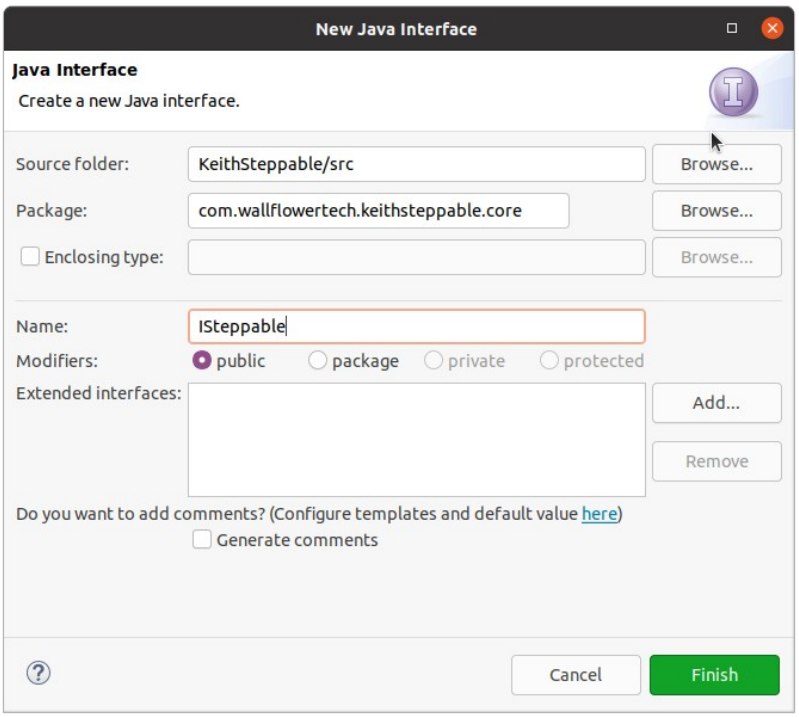
The Steppable enhancement for the Keith framework is a way to split the logic in an automated process into steps. This concept is useful for pausing a process between logical flows, stopping the logic in case of unrecoverable error, and providing functionality to either start or resume a process at a specific step.

If you are not familiar with the Keith framework, then please review the documentation before continuing. This document addresses creating the ISteppable interface then implementing it in an automated process.

Enhancement

First, create a new Java project and add the keith-[version].jar, where [version] is replaced with the jar version such as 1.0.0.





Next, create a new interface.

Using an interface is not required, but because static methods are supported in interfaces beginning in Java 1.8 and because recommended practice is to maintain constants in interfaces, it makes sense to utilize an interface here.

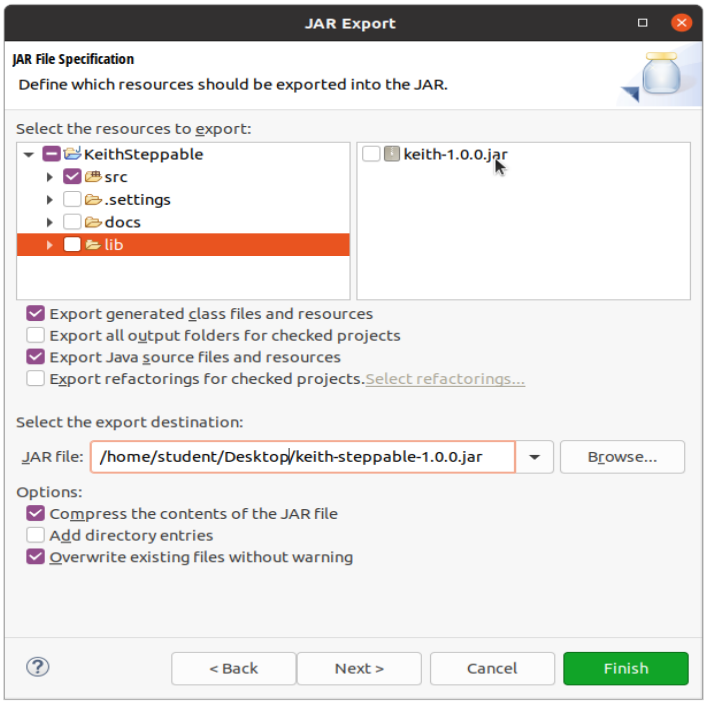
Keith Framework - Steppable

The ISteppable interface has a single constant and 3 static methods:

```
package com.wallflowertech.keithsteppable.core;
import com.wallflowertech.keith.core.AbstractApplication;
import com.wallflowertech.keith.properties.CustomProperties;

public interface ISteppable {
	static final String STEP_KEY = "step";

	static void validate() {
		CustomProperties properties =
			AbstractApplication.getApplicationProperties();
		if (!properties.containsKey(STEP_KEY)) {
			throw new NullPointerException("The step key does not exist in
				getProperties().");
		}
	}

	static String readStep() {
		return AbstractApplication.getApplicationProperties()
			.getProperty(STEP_KEY).trim();
	}

	static void writeStep(String step) {
		CustomProperties properties =
			AbstractApplication.getApplicationProperties();
		properties.put(STEP_KEY, step);
		properties.write();
	}
}
```

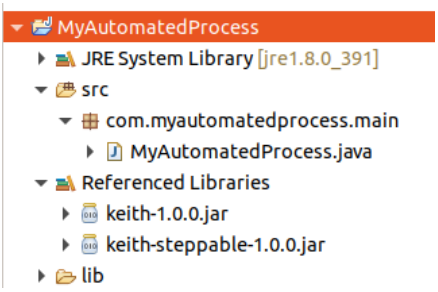
Finally, export the project as a JAR file. Insure the keith-[version].jar is excluded from the export.

In addition, include version numbers for your JAR to support future versioning.



Implementation

Add the exported enhancement JAR to the automated process classpath.



Update the code to use the enhancement:

- Implement the interface in your class:

```
public class MyAutomatedProcess extends AbstractApplication
      implements ISteppable {
```

- Add the interface constant to the getProperties() method:

```
@Override
protected Map<String, String> getProperties() {
      Map<String, String> properties = new HashMap<>();
      properties.put("check.period", "5000");
      properties.put(STEP_KEY, "1");
      return properties;
}
```

- Add code using the enhancement:

```
@Override
protected void process() throws Exception {
      ISteppable.validate(); // validate the constant exists
}

@Override
protected void run() throws Exception {
      String step = ISteppable.readStep(); // get current step
      switch (step) {
      case "1":
            // do work
            ISteppable.writeStep("2"); // update with next step
            break;
      case "2":
            // do work
            ISteppable.writeStep("3"); // update with next step
            break;
      case "3":
            // do work
            ISteppable.writeStep("1"); // update with first step
            break;
      }
}
```

This step example implementation requires the automated process run 3 times, once for each step. Sequential runs resume with the next step as stored in the external properties file.

Summary

Following the spirit of the Keith framework, an enhancement should support a common feature using as little code as possible. In other words, abstract the concept as much as possible to allow programmers creative freedom.

The ISteppable enhancement makes decisions for functionality, yet remains simple. For instance, by using a String for the step value programmers may name steps “create”, “edit”, “download”, “insert”, etc., rather than be confined to using digits had the data type been set to Integer. In addition, the ISteppable.validate() method insures the required key exists in the getProperties() method rather than requiring programmers verify the key exists in their code.

Possible enhancements may not be apparent at first, but as with other styles of applications, any time an operation is repeated in multiple automated processes, it may be useful to convert the operation to an API, then adapt the API to be used as part of the Keith framework.